
Efficient Diffusion LLMs via Temporal-Spatial Parallel Decoding and Confidence Extrapolation

Zekai Li¹ Ji Liu¹ Yiqing Huang¹ Ziqiong Liu¹ Dong Li¹ Emad Barsoum¹

Abstract

Diffusion-based large language models (dLLMs) support parallel text generation via iterative denoising, yet inference remains latency-heavy because many steps are spent on redundant refinement and repeated remasking of tokens whose final values are already determined. Prior acceleration methods mainly depend on step-local confidence heuristics or fixed schedules, which are sensitive to prompt and task variation and ignore strong positional effects within a sequence. We cast diffusion decoding as a dynamic control problem and show that token-wise denoising trajectories provide the key signal for reliable control. We propose a trace-aware decoding framework with two components. First, **Temporal-Spatial Parallel Decoding (TSPD)** uses a lightweight temporal-spatial controller that consumes per-token trajectory features, including confidence, entropy, and momentum, together with token position, to decide when a token has converged and can be safely fixed. Second, we introduce **Confidence Extrapolation (CE)**, a training-free state-space module that forecasts future logit trends with uncertainty to support proactive decisions, including safe look-ahead and targeted stabilization when trajectories are oscillatory or underconfident. Together, TSPD and CE reduce unnecessary denoising iterations while preserving output quality, and they compose cleanly with system optimizations such as KV caching.

1. Introduction

Diffusion-based large language models (dLLMs) (Nie et al., 2025; Ye et al., 2025; Khanna et al., 2025; Zhao et al., 2025) offer an alternative to autoregressive decoding (Schoenholz et al., 2024; Radford et al., 2018) by updating many to-

ken positions in parallel through iterative denoising. This framework supports flexible generation orders, infilling, and a direct quality speed trade off through the number of denoising steps (Ho et al., 2020; Song et al., 2020; Nichol & Dhariwal, 2021). Recent masked diffusion Language models (Sahoo et al., 2024; He et al., 2023; Nie et al., 2024) and LLaDA style models (Nie et al., 2025) show that diffusion can be competitive at scale. Despite this parallelism, inference latency remains high in practice. Each denoising step typically requires a full sequence forward pass under bidirectional attention (Seo et al., 2016; Wibisono & Wang, 2023; Feng et al., 2025), and many steps are spent refining tokens whose final values are already determined. Repeated remasking further amplifies this cost when uncertainty persists at a small subset of positions.

Prior work reduces this overhead by reusing computation across steps, adjusting unmasking schedules, or committing more tokens per step (Wu et al., 2025b;a; Kong et al., 2025; Wang et al., 2025c; Israel et al., 2025). These methods improve throughput, but most commit and stopping decisions still depend on step local heuristics, fixed schedules, or global thresholds (Li et al., 2025; Chen et al., 2025b;a). Such signals are brittle because diffusion confidence is not uniformly calibrated across prompts, domains, and token positions. Empirically, diffusion decoding exhibits structured temporal and spatial behaviors, including delayed stabilization, underconfident but correct tokens, and oscillatory confidence (Huang et al., 2025; Wang et al., 2025b). A single step snapshot or heuristic rule cannot separate these regimes. As a result, local rules often pay for redundant denoising after convergence, or they commit prematurely and lose quality.

In this work, we treat diffusion decoding as a *sequential control problem* rather than a sequence of independent threshold tests. A key observation is that denoising produces token wise *trajectories* across steps, such as confidence and entropy traces, whose temporal patterns reveal both correctness and stability. This shifts the question from “is the token confident at step t ” to “does the trajectory indicate convergence, and how will it change if we continue denoising”. Answering this requires a temporal-spatial-aware controller

¹Advanced Micro Devices, Inc. (AMD). Correspondence to: Ji Liu <liuji@amd.com>, Ziqiong Liu <ziqioliu@amd.com>.

that can interpret patterns such as steady growth, oscillation, and delayed stabilization, as well as a mechanism that can exploit predictable trends to avoid unnecessary steps.

We propose **Temporal-Spatial Parallel Decoding (TSPD)**, a temporal-spatial-aware framework that improves both efficiency and accuracy. TSPD introduces a lightweight *temporal-spatial controller* that predicts whether a position is ready to be fixed using sequential trace features (confidence, entropy, and momentum over recent steps) together with relative spatial position within a block. A shared sequential perception model, captures dynamics that step local classifiers and hand tuned thresholds often miss, improving robustness across prompts and positional effects. To reduce waiting time when confidence is rising but not yet decisive, we further introduce a training-free and plug-in **Confidence Extrapolation mechanism (CE)**. CE models confidence evolution as a simple state space process and forecasts near future confidence together with uncertainty. A risk-aware horizon rule limits look ahead based on left context progress and forecast uncertainty, so extrapolation is applied only when supported by reliable history.

We evaluate our framework (TSPD and CE) on LLaDA-8B-Instruct (Nie et al., 2025) across four benchmarks spanning arithmetic reasoning and code generation, including GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), HumanEval (Chen, 2021), and MBPP (Austin et al., 2021b). Across multiple dLLM backbones and generation settings, our method delivers a consistently stronger speed quality tradeoff than state of the art baselines. On GSM8K with 256 token generation, it achieves $5.0\times$ speedup without cache, and compounds to $11.2\times$ when combined with KV caching, showing that our controller level gains are complementary to system optimizations. The advantage increases with longer generation: at 1024 tokens, our approach attains 64.1 Tokens Per Second (TPS) and up to $58.3\times$ speedup, while keeping accuracy essentially unchanged and in some cases slightly improved. In summary, our contributions are threefold:

- We propose **TSPD**, a temporal spatial aware decoding controller that uses temporal confidence dynamics and positional context to make robust token fixing decisions, mitigating the brittleness of step local heuristics.
- We introduce a training free and plug and play **confidence extrapolation** module that forecasts near future confidence under a state space model with quantified uncertainty, and applies risk aware look ahead to complement diverse decoding controllers.
- We provide extensive evaluations and ablations on a wide range of dLLMs, task domains, and generation settings, demonstrating substantial speedups with near lossless accuracy and full compatibility with KV cache.

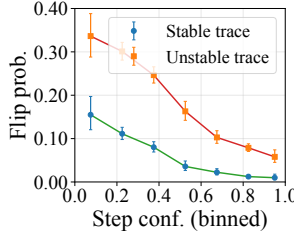


Figure 1. Same local confidence with different trace can lead to different decoding results.

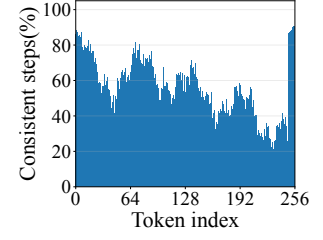


Figure 2. Tokens at more rightward spatial positions tend to stabilize later.

2. Related Work

2.1. Diffusion-Based Large Language Models

Diffusion models (Sohl-Dickstein et al., 2015; Song & Ermon, 2019) have been extended from continuous data to discrete sequences, enabling non-autoregressive generation via iterative denoising. Early discrete diffusion work established Markov formulations in categorical spaces, including multinomial diffusion (Hoogeboom et al., 2021) and D3PM (Austin et al., 2021a). Continuous-time views (Campbell et al., 2022) further unify discrete diffusion and support flexible sampling schedules. SEDD (Lou et al., 2023) improves learning by modeling ratios of marginal probabilities with a score-entropy objective, narrowing the gap to autoregressive language models (LMs).

Building on these foundations, masked dLLMs have become a practical paradigm for text generation. MDLM (Sahoo et al., 2024) shows that a simplified masked-diffusion recipe can substantially improve perplexity and sampling quality. At scale, diffusion LLMs are developed either by adapting strong autoregressive backbones (e.g., DiffuLLaMA) (Gong et al., 2024) or by training from scratch (e.g., LLaDA) (Nie et al., 2025), achieving competitive instruction following and reasoning. Dream 7B (Ye et al., 2025) further advances open diffusion LLMs and highlights diffusion-specific capabilities such as arbitrary-order generation and infilling with tunable quality-speed trade-offs.

2.2. Decoding Acceleration for Diffusion LLMs

Despite parallel updates, diffusion decoding can be latency-heavy because each denoising step typically requires a full-sequence forward pass under bidirectional attention. Prior work improves efficiency via three main directions: (i) computation reuse and caching, (ii) better parallel commitment or sampling schedules, and (iii) plug-in accelerators.

On computation reuse, dLLM-Cache (Liu et al., 2025) caches static prompts and selectively updates stable tokens; Fast-dLLM (Wu et al., 2025b) introduces block-wise approximate KV caching with confidence-aware parallel decoding; DInfer (Ma et al., 2025) modularizes the inference pipeline and reports $\sim 10\times$ speedup over Fast-dLLM with-

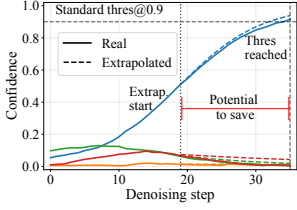


Figure 3. Missed acceleration opportunity of passive waiting compared to lookahead.

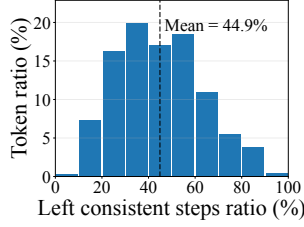


Figure 4. Distribution of consistent steps ratio that can be extrapolated.

out quality loss; Fast-dLLM v2 (Wu et al., 2025a) further supports block-diffusion adaptation of pretrained autoregressive backbones and hierarchical KV caching, achieving up to $2.5\times$ speedup over standard autoregressive decoding. On algorithmic decoding, SlowFast Sampling (Wei et al., 2025) alternates conservative exploration and accelerated decoding; Local Leap (Kong et al., 2025) exploits local determinism around high-confidence anchors; Learn2PD (Bao et al., 2025) learns a parallel decoding policy with strong speedups on LLaDA; and adaptive parallel decoding (Israel et al., 2025) dynamically controls parallelism and uses a small autoregressive verifier for a controllable speed–quality trade-off. Training-free plug-ins include FlashDLM (Hu et al., 2025) (FreeCache + guided diffusion), ES-dLLM (Anonymous, 2026) (early skipping via convergence signals), and foreseeing movement decoding (Mo et al., 2025).

Most existing methods still make decisions from per-step heuristics or fixed schedules and respond to uncertainty by passively running more denoising steps. In contrast, we emphasize that diffusion decoding exposes informative *token-wise trajectories* (confidence/probability traces) that enable more input-adaptive control. Accordingly, we develop (i) a temporal–spatial correctness sensor that fuses history features with positional cues to infer stabilization, and (ii) an *active forecasting* module that extrapolates near-future confidences with calibrated uncertainty (e.g., via state-space/Kalman-style models (Kalman, 1960)) for risk-limited look-ahead decisions. This trajectory-aware perspective complements caching and sampler optimizations and targets finer-grained speed–quality control for diffusion LLM inference.

3. Methods

3.1. Preliminary

3.1.1. PARALLEL DECODING OF DIFFUSION LLMs

Diffusion LLMs (dLLMs) generate a length- L sequence $x_0 \in \{0, 1, \dots, V-1\}^L$ by iteratively denoising a masked version of the sequence. Let $\alpha \in [0, 1]$ denote the masking ratio (noise level) to avoid overloading t as “time”. The forward process independently replaces each token with the

mask token m with probability α :

$$q_\alpha(x^\alpha | x) = \prod_{i=1}^L [(1-\alpha)\mathbf{1}[x_i^\alpha = x_i] + \alpha\mathbf{1}[x_i^\alpha = m]]. \quad (1)$$

where x_α is the corrupted sequence at masking ratio α . The reverse process denoises in K discrete steps by repeatedly predicting a token distribution for masked positions. Given a prompt $c = (c_1, \dots, c_M)$ and current masked response $y^{(k)}$, a mask predictor p_θ outputs per-position distributions and we form a provisional sequence by greedy decoding:

$$\hat{y}_i^{(k)} = \arg \max_{v \in [V]} p_\theta(v | c, y^{(k)}), \quad i \in [L]. \quad (2)$$

and remark the n_k positions with the smallest c_i , where n_k (equivalently the masking ratio) follows a predefined schedule that decreases over steps. This parallel denoising–remasking loop yields the final response after K iterations.

3.1.2. EARLY STOPPING FOR PARALLEL DECODING

Diffusion-based LLMs (dLLMs) generate text by iteratively denoising a masked sequence. Although additional steps can refine quality, many tokens reach their final values well before the last denoising step, so continuing refinement wastes compute and increases latency. Early stopping accelerates inference by committing tokens once further updates are unlikely to change the final outcome.

Existing early-stopping methods are dominated by *heuristic or step-local* controllers. Heuristic rules (e.g., Fast-dLLM (Wu et al., 2025b), Prophet (Li et al., 2025), Credit Decoding (Wang et al., 2025a)) apply global thresholds on current-step statistics such as top-1 confidence or entropy, while learned controllers (e.g., Learn2PD (Bao et al., 2025)) train a lightweight predictor but still make thresholded accept/reject decisions at inference. We show that this step-local, uniformly-applied design is unreliable due to the temporal and spatial structure of diffusion trajectories, and that passive waiting misses forecastable acceleration opportunities.

Step-local and Spatial-uniform Control Is Unreliable.

The same current-step confidence can correspond to very different future outcomes, and the ambiguity varies across token positions. We validate this on a held-out C4 (Raffel et al., 2020) subset by extracting token-states from intermediate denoising steps. For position i at step k , let $p_i^{(k)} \in \Delta^V$ be the predictive distribution; we group token-states with nearly indistinguishable step-local distributions (matched by entropy and tight Jensen–Shannon divergence) but different future trajectories. We define the flip probability $P_{\text{flip}}(k) = \Pr[\hat{y}_{0,i}^{(K)} \neq \hat{y}_{0,i}^{(k)}]$ and observe that, within the same confidence bin, unstable traces have multiple- $\times$ higher flip rates than stable ones (Fig. 1). Moreover, stabilization is

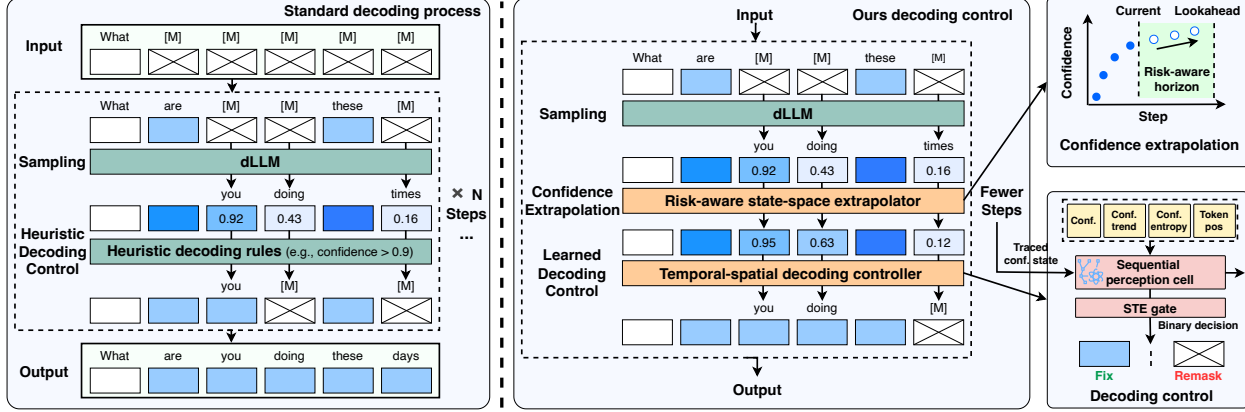


Figure 5. Comparison between standard dLLM parallel decoding (left) and our framework (right). Standard decoding applies step-local heuristics after each full-sequence denoising pass, often revisiting already-correct tokens. Our framework inserts a risk-aware confidence extrapolator (CE) and a temporal-spatial decoding controller (TSPD): CE provides uncertainty-aware look-ahead confidence, and TSPD uses token-wise confidence traces and position context to fix stable tokens. This reduces redundant steps while preserving quality.

position-dependent: tokens at more rightward positions stabilize later (Fig. 2), so no single global threshold (heuristic or learned) can be well-calibrated for all positions simultaneously.

Heuristic Controllers Are Fragile. Most prior methods ultimately threshold a step-local score with a fixed τ . This includes heuristic rules and learned filters such as Learn2PD (Bao et al., 2025), which replace hand-crafted scores with a learned score $f_\theta(\cdot)$ but still decide “accept” if $f_\theta(\cdot) > \tau$. When the score–stability relationship shifts across steps or positions, a static τ becomes either overly conservative (hurting speed) or overly aggressive (hurting quality), making tuning sensitive and workload-dependent.

Passive Waiting Misses forecastable Acceleration. When uncertain, existing controllers typically wait for additional denoising steps until a threshold is crossed. However, once a token enters a trend-consistent regime, its future confidence is often predictable from a short prefix. Fig. 3 shows a case where the winning token follows a stable upward trend long before reaching a standard confidence threshold, suggesting that a look-ahead mechanism can commit earlier. Fig. 4 further indicates that many tokens exhibit trend-consistent portions, leaving substantial headroom for saving steps.

These findings motivate two principles: (i) early stopping should be *temporal- and position-aware*, leveraging token-wise trajectories and spatial heterogeneity rather than step-local uniform thresholds; and (ii) it should be *proactive*, using predictable dynamics with calibrated uncertainty to anticipate stabilization instead of passively waiting. As shown in Fig. 5, we instantiate these principles with (i) TSPD, a trajectory-based controller that models token history and relative position, and (ii) a training-free confidence extrapolation module that forecasts near-future confidences with uncertainty to enable risk-limited look-ahead decisions.

3.2. Temporal-Spatial Parallel Decoding (TSPD)

In practice, token-wise uncertainty evolves with diverse trajectories (e.g., monotone convergence, oscillation, or delayed stabilization), and these patterns are not reliably inferred from a single-step snapshot. We therefore develop *Temporal-Spatial Parallel Decoding (TSPD)*, a lightweight learnable controller that tracks per-token temporal traces and spatial position within each decoding block, and outputs a binary *fix-or-continue* decision for every token at every diffusion step.

Temporal-spatial Decoding Signals. Let $x^{(t)} = (x_1^{(t)}, \dots, x_L^{(t)})$ denote the intermediate sequence at diffusion step t (with t decreasing from T to 0), and let $p_\theta(\cdot | c, x^{(t)})$ be the dLLM token distribution. For each position i , we take the top-1 candidate $\hat{x}_i^{(t)} = \arg \max_v p_i^{(t)}(v)$ and its confidence $p_i^{(t)} = \max_v p_i^{(t)}(v)$, and measure uncertainty by the predictive entropy $H_i^{(t)} = -\sum_v p_i^{(t)}(v) \log p_i^{(t)}(v)$. To reduce step-wise noise, we track an exponential moving average confidence $\bar{p}_i^{(t)} = \alpha p_i^{(t)} + (1 - \alpha) \bar{p}_i^{(t+1)}$ and use its one-step difference as a momentum feature $\Delta \bar{p}_i^{(t)} = \bar{p}_i^{(t)} - \bar{p}_i^{(t+1)}$. We also include the token’s relative position within the block, $\phi(i) = i/(L - 1)$. Thus, at each step t TSPD constructs the per-token feature vector as:

$$r_i^{(t)} \triangleq [p_i^{(t)}, H_i^{(t)}, \Delta \bar{p}_i^{(t)}, \phi(i)], \quad (3)$$

which captures instantaneous evidence $(p_i^{(t)}, H_i^{(t)})$, temporal trend $(\Delta \bar{p}_i^{(t)})$, and spatial context $(\phi(i))$.

Sequential binary controller. TSPD employs a shared recurrent controller to maintain a hidden state for each token position, enabling memory over the denoising trajectory. Specifically, for each token i we update

$$h_i^{(t)} = f_\theta(h_i^{(t+1)}, r_i^{(t)}), \quad (4)$$

Algorithm 1 Inference Workflow

Require: dLLM $\mathcal{M}$ with K steps; prompt x ; generation length L ; controller TSPD; extrapolator CE.

- 1: $X \leftarrow \text{concat}(x, [\text{MASK}]^L)$; $F \leftarrow \mathbf{0}$ $\triangleright F_j=1$ means token j is fixed
- 2: Initialize per-token trace buffer $\mathcal{H}$
- 3: **for** $t = K, \dots, 1$ **do**
- 4: $p(\cdot) \leftarrow \mathcal{M}(X)$ $\triangleright$ one dLLM forward pass
- 5: Update trace history $\mathcal{H} \leftarrow \text{update}(\mathcal{H}, p)$
- 6: $\hat{r} \leftarrow \text{CE}(\mathcal{H}, F)$
- 7: $A \leftarrow \text{TSPD}(\hat{r}, \text{pos}, F)$ $\triangleright$ binary fix decisions
- 8: $X \leftarrow \text{commit}(X, p, A)$; $F \leftarrow F \vee A$
- 9: **if** $\sum F = L$ **then break**
- 10: **end if**
- 11: $X \leftarrow \text{remask}(X, p, F)$
- 12: **end for**
- 13: **return** $\text{resp}(X)$

where $f_\theta(\cdot)$ is a lightweight recurrent update (e.g., a GRU (Cho et al., 2014)/LSTM (Hochreiter & Schmidhuber, 1997) cell) with parameters ψ . A linear head produces a gate logit

$$z_i^{(t)} = Wh_i^{(t)} + b, \quad (5)$$

which is converted to a soft fixing propensity $\pi_i^{(t)} = \sigma(z_i^{(t)})$. Unlike threshold-tuned score predictors, TSPD executes a *direct binary action* by hard gating:

$$a_i^{(t)} \in \{0, 1\}, \quad a_i^{(t)} = \mathbb{I}[\pi_i^{(t)} \geq 0.5], \quad (6)$$

where $a_i^{(t)} = 1$ indicates that token i is *fixed* (committed and excluded from further denoising) and $a_i^{(t)} = 0$ keeps it active.

Training by Oracle Imitation with Discrete Decisions.

We supervise TSPD using an oracle fixing policy that marks a token as fixable once its candidate token matches the final target at that step. Formally, let $y_i^{(t)} \in \{0, 1\}$ denote the oracle *fix* label for token i at step t . We train the controller via binary cross-entropy on logits:

$$\mathcal{L}_{\text{gate}} = \frac{1}{LT} \sum_{t=0}^T \sum_{i=1}^L \text{BCEWithLogits}(z_i^{(t)}, y_i^{(t)}). \quad (7)$$

To align training with inference-time hard gating, we use a straight-through estimator (STE): the forward pass applies the hard decision $a_i^{(t)}$ to simulate fixing, while gradients are backpropagated through the continuous $\pi_i^{(t)}$ (equivalently, through $z_i^{(t)}$ in (7)). By jointly leveraging temporal traces and spatial position, TSPD provides robust parallel decoding control with negligible overhead.

3.3. Confidence Extrapolation

Most existing decoding accelerations are passive: when confidence is low, the model simply runs more denoising steps until the distribution sharpens. We instead propose *Confidence Extrapolation* (CE), a training-free, plug-in module that *forecasts near-future confidence* from the observed confidence trace and feeds a calibrated, risk-aware confidence signal to any downstream decoding controller (e.g., TSPD, Learn2PD, or threshold rules). CE does not modify the base diffusion model parameters and can be combined with existing controllers without changing their decision logic.

State-space Extrapolator on Confidence. For each position j in a decoding block, let $c_t(j) \in [0, 1]$ denote the confidence at diffusion step t (e.g., top-1 probability or a margin-based confidence). To obtain a stable time series, we apply a monotone transform $y_t(j) = \phi(c_t(j))$ (e.g., log-odds). We model $y_t(j)$ as a noisy observation of a latent confidence level:

$$y_t(j) = \ell_t(j) + n_t, \quad n_t \sim \mathcal{N}(0, R_t(j)). \quad (8)$$

CE maintains a level-velocity latent state

$$x_t(j) = [\ell_t(j), \dot{\ell}_t(j)]^\top, \quad x_{t+1}(j) = Ax_t(j) + w_t, \quad (9)$$

where Q determines how rapidly the underlying trend is allowed to change, and A is a fixed two-by-two transition matrix with ones on the diagonal and the upper-right entry, and zeros elsewhere. Given the filtered estimate $(\hat{x}_t, P_t)$, the h -step forecast is

$$\hat{x}_{t+h|t} = A^h \hat{x}_t, \quad P_{t+h|t} = A^h P_t (A^h)^\top + Q_h, \quad (10)$$

where $Q_h \triangleq \sum_{i=0}^{h-1} A^i Q (A^i)^\top$ is the accumulated process noise over h steps. And the forecast variance of the confidence level is $\sigma_{t+h}^2(j) = (P_{t+h|t}(j))_{1,1}$. We map forecasts back to confidence by $\hat{c}_{t+h}(j) = \phi^{-1}(\hat{\ell}_{t+h}(j))$.

Risk-aware Horizon Selection. CE selects the *furthest* look-ahead horizon that is both (i) supported by a reliable history window and (ii) safe under extrapolation uncertainty. Although denoising within a block is parallel, later positions become reliable only after sufficient left context is fixed. We use $[x]_0^1 \triangleq \min\{1, \max\{0, x\}\}$ to clamp x into $[0, 1]$. Let $\text{left_coverage}_t(j) \in [0, 1]$ be the fraction of fixed positions left of j at step t , and define an extrapolation potential

$$r_t(j) = \left[\frac{\text{left_cvg}_t(j) - \tau}{1 - \tau} \right]_0^1, \quad h_{\text{pot}} = H r_t(j). \quad (11)$$

where τ is a coverage threshold and H is the maximum look-ahead. For a candidate horizon $h \leq h_{\text{pot}}$, we form a conservative lower bound in the transformed space:

$$\text{LB}_{t+h} \triangleq \hat{\ell}_{t+h} - z \sigma_{t+h}, \quad \hat{c}_{t+h}^{\text{LB}} = \phi^{-1}(\text{LB}_{t+h}). \quad (12)$$

Table 1. Benchmark results on the LLaDA-8B-Instruct suite with a generation length of 256. Performance is measured using TPS, speedup, and accuracy ($\dagger$ denotes without KV cache).

Task	Methods	Inference Efficiency		Acc.
		TPS $\uparrow$	Speed $\uparrow$	
GSM8K (5-shot)	Vanilla	6.9	1.0 $\times$	79.3
	F-dLLM	53.8	7.8 $\times$	78.5
	CD	54.5	7.9 $\times$	78.7
	Prophet $\dagger$	11.0	1.6 $\times$	79.4
	Learn2PD $\dagger$	29.0	4.2 $\times$	79.1
	Ours $\dagger$	34.2	5.0 $\times$	79.4
	Ours	77.3	11.2$\times$	78.8
MATH (4-shot)	Vanilla	9.2	1.0 $\times$	33.5
	F-dLLM	51.5	5.6 $\times$	33.2
	CD	50.6	5.5 $\times$	33.1
	Prophet $\dagger$	15.7	1.7 $\times$	32.7
	Learn2PD $\dagger$	29.4	3.2 $\times$	32.3
	Ours $\dagger$	36.8	4.0 $\times$	33.0
	Ours	76.4	8.3$\times$	33.2
HumanEval (0-shot)	Vanilla	6.5	1.0 $\times$	41.5
	F-dLLM	24.1	3.7 $\times$	43.3
	CD	24.7	3.8 $\times$	43.7
	Prophet $\dagger$	7.8	1.2 $\times$	41.5
	Learn2PD $\dagger$	23.4	3.6 $\times$	40.3
	Ours $\dagger$	27.3	4.2 $\times$	42.5
	Ours	53.3	8.2$\times$	43.0
MBPP (3-shot)	Vanilla	5.8	1.0 $\times$	29.4
	F-dLLM	41.8	7.2 $\times$	28.2
	CD	40.6	7.0 $\times$	29.3
	Prophet $\dagger$	8.1	1.4 $\times$	29.2
	L2PD $\dagger$	29.6	5.1 $\times$	29.2
	Ours $\dagger$	32.5	5.6 $\times$	29.5
	Ours	68.4	11.8$\times$	29.2

where z controls risk tolerance. For brevity, we write $\hat{\ell}_{t+h} \equiv \hat{\ell}_{t+h|t}$ and omit the position index j when clear. We choose

$$h^{(t,j)} = \max\{h \leq h_{\text{pot}}(t,j) : \hat{c}_{t+h}^{\text{LB}}(j) \geq c_{\min}\}. \quad (13)$$

and fall back to $h^{(t,j)} = 0$ if no horizon passes. The controller then consumes $\tilde{c}_{t(j)} = \hat{c}_{t+h^{(t,j)}}^{\text{LB}}(j)$ in place of the raw $c_{t(j)}$.

Integration with Decoding Controllers. CE exposes a minimal interface that outputs $\tilde{c}_{t(j)}$ (and optionally $h^{(t,j)}$) per step and position, making it directly compatible with threshold-based or learned controllers. As shown in Alg. 1, we simply replace the controller’s per-step confidence feature with $\tilde{c}_t(j)$, enabling earlier commitment when the forecast indicates a reliably increasing confidence trajectory.

Table 2. Benchmark results on the Dream-7B and LLaDA-MoE suite with a generation length of 256. Performance is measured using TPS (tokens/sec), speedup, and accuracy ($\dagger$ denotes without KV cache).

Task	Methods	TPS $\uparrow$	Speed $\uparrow$	Acc.
Dream-7B	Vanilla	9.1	1.0 $\times$	75.0
	F-dLLM	48.2	5.3 $\times$	74.8
	CD	42.8	4.7 $\times$	74.5
	Prophet $\dagger$	15.1	1.7 $\times$	74.9
	Learn2PD $\dagger$	19.1	2.1 $\times$	73.4
	Ours $\dagger$	28.2	3.1 $\times$	75.3
	Ours	69.2	7.6$\times$	75.0
LLaDA-MoE	Vanilla	4.2	1.0 $\times$	75.2
	F-dLLM	9.7	2.3 $\times$	75.3
	CD	10.5	2.5 $\times$	75.2
	Prophet $\dagger$	6.7	1.6 $\times$	75.0
	Learn2PD $\dagger$	13.0	3.1 $\times$	75.3
	Ours $\dagger$	13.8	3.3 $\times$	75.4
	Ours	21.2	5.1$\times$	75.2

4. Experiments

4.1. Experimental Settings

Models and Datasets. We mainly implement our method on the representative dLLM: LLaDA-8B-Instruct (Nie et al., 2025) to measure the acceleration of the inference process across various benchmarks. To ensure the broad applicability of the method, we conduct experiments on four datasets covering three different types of problems, which are GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), HumanEval (Chen, 2021), and MBPP (Austin et al., 2021b). To demonstrate the generalizability of our method across diverse dLLM architectures, we conduct additional comparisons with existing methods on Dream-7B (Ye et al., 2025) and LLaDA-MoE (Zhu et al., 2025), further confirming the effectiveness of our approach. In this paper, we primarily compare our method with state-of-the-art dLLM acceleration approaches, including Fast-dLLM (Wu et al., 2025b), Credit Decoding (CD) (Wang et al., 2025a), Prophet (Li et al., 2025), and Learn2PD (Bao et al., 2025). Among these methods, Fast-dLLM and Credit Decoding leverage Key-Value (KV) caching, whereas Learn2PD and Prophet do not utilize KV cache mechanisms.

TSPD Training. To train the TSPD model that can be applied to a wide range of tasks, we select 40 samples from each of the 66 types of questions in the FLAN dataset (out of the used evaluation benchmarks), resulting in a total of 2,640 samples for training. In this experiment, we adopt the simplest two-layer LSTM as our filter model. Since the dLLMs remain frozen and only f_θ is trained, the number of trainable parameters is extremely limited. For example, for an LLaDA with a block size of 32, the total number of trainable parameters is only 2,112. We train the f_θ for 5,000

Table 3. Benchmark results on the LLaDA-8B-Instruct suite with a generation length of 512 and 1024. † denotes without KV cache.

Gen Len.	Methods	TPS↑	Speed↑	Acc.
512	Vanilla	3.2	1.0×	77.5
	Fast-dLLM	35.3	11.0×	77.2
	Learn2PD†	39.4	12.3×	77.8
	Ours†	45.4	14.2×	77.9
	Ours	79.0	24.7×	77.6
1024	Vanilla	1.1	1.0×	77.0
	Fast-dLLM	21.6	19.6×	74.7
	Learn2PD†	24.9	22.6×	78.7
	Ours†	27.3	24.8×	79.3
	Ours	64.1	58.3×	78.5

epochs until the model converged. The learning rate is set to 0.001, and the AdamW optimizer is used to optimize f_θ .

Our training process consists of two stages. In the first stage, samples are collected by following an Extremely Greedy Parallel (Bao et al., 2025) policy, recording the confidence scores and token selections at each step during parallel decoding. This data is then used in the second stage to train the TSPD model f_θ . The data collection in the first stage is conducted on 2 NVIDIA RTX A100 GPUs and takes approximately two hours. The subsequent training of the TSPD model in the second stage is deployed on the same GPU and required only 6 minutes. The details of training are provided in Appendix A.4.

Evaluation. We evaluate the inference acceleration and generation quality of TSPD and CE methods using quantitative metrics. The inference speed is quantified with Tokens Per Second (TPS), indicating the average number of tokens generated per second. The generation quality is measured using task-specific metrics, such as accuracy for GSM8K, reflecting the model’s performance under acceleration. In addition, we set the generation length to 256 and 1024, and the block size to 32.

4.2. Main Results

Overall Performance. Table 1 reports results on the LLaDA-8B-Instruct suite with generation length 256. Across GSM8K, MATH, HumanEval, and MBPP, our method consistently achieves the highest throughput and speedup while preserving accuracy close to Vanilla. With KV cache, our method reaches **77.3 TPS / 11.2×** on GSM8K, **76.4 / 8.3×** on MATH, **53.3 / 8.2×** on HumanEval, and **68.4 / 11.8×** on MBPP. Without cache, our method still outperforms Prophet and Learn2PD with same settings on all tasks (e.g., 34.2 vs. 29.0 TPS on GSM8K, 36.8 vs. 29.4 on MATH), demonstrating that our gains do not rely on additional caching.

Performance over Different dLLMs. Table 2 shows that

Table 4. A comparison of our method with and without KV Cache. The results show a significant performance improvement when augmented with both Dual and Prefix Caches, underscoring that our method is orthogonal to and fully compatible with existing KV caching strategies.

Methods	TPS↑	Speed↑	Acc.
TSPD & CE	34.2	5.0×	79.4
+ Dual Cache	77.3	11.2×	78.8
+ Prefix Cache	37.3	5.4×	79.3

Table 5. Ablation of the effectiveness of TSPD and CE modules using LLaDA-8B-Instruct model on GSM8K benchmark at generation length of 256 tokens. † denotes without KV cache.

Methods	TPS↑	Speed↑	Acc.
Ours†	34.2	5.0×	79.4
w.o. TSPD	20.0	2.9×	79.1
w.o. CE	30.3	4.4×	79.5

the improvements transfer across backbones. On Dream-7B, our method achieves **69.2 TPS / 7.6×** with accuracy matching Vanilla (75.0%), and our method (without KV cache) remains competitive (28.2 TPS / 3.1×). On LLaDA-MoE, our method improves throughput to **21.2 TPS / 5.1×** with essentially unchanged accuracy (75.2%), indicating that CE and the controller are not model-specific and generalize to MoE architectures.

Performance over Different Generation Lengths. Table 3 evaluates longer generations (512/1024). Speedups increase with length: at 512 tokens, our method achieves **79.0 TPS / 24.7×**; at 1024 tokens, it reaches **64.1 TPS / 58.3×**, while maintaining accuracy comparable to Vanilla. In contrast, Fast-dLLM shows a larger accuracy drop at 1024 (from 77.0% to 74.7%). Overall, our method scales favorably with generation length, yielding larger end-to-end acceleration without sacrificing generation quality.

4.3. Compatibility with Key-Value Cache

We evaluate compatibility with KV caching (Pope et al., 2023; Liu et al., 2024) by integrating Dual Cache (Wu et al., 2025b) and Prefix Cache (Wu et al., 2025b) on GSM8K (256 tokens). As shown in Table 4, the baseline TSPD+CE achieves 34.2 TPS (5.0×) with 79.4% accuracy. Adding Dual Cache boosts throughput to 77.3 TPS (11.2×) with a modest accuracy drop (78.8%), while Prefix Cache improves to 37.3 TPS (5.4×) with 79.3% accuracy. Overall, our method is orthogonal to standard KV caching and can directly benefit from these techniques for further speedups.

Table 6. A comparison of the acceleration performance using filter models of varying complexity. The results indicate that a two-layer LSTM model achieves the optimal balance by providing significant speedup and maintaining high accuracy.

# Layers	TPS↑	Speed↑	Acc.
1-layer MLP	29.7	4.3×	78.7
2-layer MLP	30.4	4.4×	78.8
4-layer MLP	26.8	3.9×	79.0
1-layer LSTM	29.0	4.2×	79.2
2-layer LSTM	30.3	4.4×	79.5
4-layer LSTM	28.3	4.1×	79.5

Table 7. A ablation of the effectiveness of the features used for TSPD using LLaDA-8B-Instruct model on GSM8K benchmark at generation length of 256 tokens.

Methods	TPS↑	Speed↑	Acc.
TSPD	30.3	4.4×	79.5
w.o. confidence	19.3	2.8×	78.8
w.o. entropy	28.3	4.1×	79.0
w.o. momentum	29.7	4.3×	79.2
w.o. position	31.1	4.5×	78.5

4.4. Sensitivity Analysis

Parameter Budget on TSPD. We vary the TSPD parameter budget within the same 2-layer LSTM architecture by scaling the hidden size. As shown in Table 8, the current **2K** setting provides the best speed–accuracy tradeoff. Reducing the budget to **1K** slightly improves TPS (30.3 → 30.9) but lowers accuracy (79.5 → 79.2). Increasing the budget to **4K** or **8K** brings at most marginal accuracy gain, while throughput drops more noticeably. This suggests that the current controller is already near the sweet spot: smaller models underfit the temporal traces, whereas larger ones add complexity with little benefit.

Training Sensitivity to Data Collection Policy. We compare EGP-collected, standard parallel-decoding, and mixed training traces under the same test-time setting. As shown in Table 9, EGP gives the best tradeoff (**30.3 TPS, 79.5 Acc**), while standard traces are much slower (**7.5 TPS**) with similar accuracy; the mixed policy is in between (**16.1 TPS, 79.5 Acc**). This suggests that EGP does not cause harmful distribution shift, but instead provides more informative borderline states for learning when to safely fix tokens.

Sensitivity to Training Data Scale.

We evaluate TSPD with different numbers of FLAN training prompts while keeping the controller architecture and decoding settings fixed. As shown in Table 10, TSPD is already close to convergence with **1320** samples: compared with the default **2640**-sample setting, the accuracy gap is

Table 8. Impact of parameter budget on TSPD performance under the same 2-layer LSTM architecture.

Param budget	Hidden size	TPS↑	Acc. ↑
1K	8	30.9	79.2
2K	12	30.3	79.5
4K	17	29.8	79.5
8K	25	26.8	79.7

Table 9. Training sensitivity of data collection policy.

	EGP	Standard PD	Mixed
TPS ↑	30.3	7.5	16.1
Acc ↑	79.5	79.6	79.5

only **0.1**, and further increasing the data to **3960** samples brings negligible gains. This is expected because TSPD is a lightweight controller trained on compact trajectory features rather than full token logits.

Sensitivity to Training Data Distribution.

We evaluate whether TSPD is sensitive to the difficulty distribution of training traces, measured by the average *stabilization-step ratio*, i.e., the fraction of denoising steps where a token already matches its final prediction. Lower ratios indicate harder traces with later stabilization, while higher ratios indicate easier traces with earlier stabilization. As shown in Table 11, TSPD is stable across different distributions, with variations within **1.1 TPS** and **0.5 Acc**. Harder traces make the controller slightly more conservative and improve accuracy, while easier traces make it more aggressive and improve TPS. Our default mixed distribution (~ 0.45) gives the best overall tradeoff.

Sensitivity to CE Hyper-parameters. We evaluate the two key CE hyper-parameters, the coverage threshold τ and the maximum look-ahead horizon H , while keeping all other settings fixed. The masking ratio α belongs to the base dLLM denoising schedule rather than our method, and is therefore not tuned here. As shown in Table 12, CE shows a stable speed–accuracy tradeoff: smaller τ or larger H makes CE more aggressive and improves TPS, while larger τ or smaller H better preserves accuracy. We use $\tau = 0.6$ and $H = 20$ by default.

4.5. Ablation Study

Effectiveness of TSPD and CE. As shown in Tab. 5, on GSM8K with generation length 256, both TSPD and CE improve throughput with negligible accuracy change. Vanilla decoding runs at 6.9 TPS with 79.3% accuracy. Removing TSPD drops throughput to 20.0 TPS (2.9×), indicating that

Table 10. Sensitivity to TSPD training data scale.

Training Samples	TPS $\uparrow$	Acc. $\uparrow$
Vanilla	6.9	79.3
330	27.1	78.8
660	28.7	79.1
1320	30.0	79.4
2640	30.3	79.5
3960	30.2	79.5

Table 11. Sensitivity to training data distribution.

Stabilization Ratio	TPS $\uparrow$	Acc. $\uparrow$
Vanilla	6.9	79.3
~ 0.25	29.6	79.7
~ 0.45	30.3	79.5
~ 0.50	30.1	79.5
~ 0.75	30.7	79.2

trace-aware token fixing provides the main speedup. Removing CE yields 30.3 TPS (4.4 $\times$), showing that confidence extrapolation brings additional gains beyond TSPD alone. Combining both achieves the best throughput, 34.2 TPS (5.0 $\times$), while maintaining comparable accuracy (79.4%).

Complexity and Overhead Analysis. Table 6 shows that moderate-capacity filters perform best. While a two-layer MLP achieves the highest efficiency among MLP variants (30.4 TPS, 4.4 $\times$ speedup), a two-layer LSTM strikes a more favorable balance, delivering substantial speedup while maintaining high accuracy (30.3 TPS, 4.4 $\times$ speedup, and 79.5% accuracy). Since deeper variants yield diminishing returns, we adopt the two-layer LSTM as the default. TSPD and CE operate only on lightweight confidence features (per-position scalars and short traces), so their runtime overhead is negligible compared to a diffusion model forward. Profiling on GSM8K (256 tokens) shows that the per-step latency of TSPD and CE is only a small fraction of one base-model forward by 0.30% and 0.13%, respectively. Thus, the observed speedups primarily come from reducing redundant denoising steps rather than shifting cost to the controller.

Ablation of the Features Used for TSPD. Table 7 ablates TSPD features, removing *confidence* causes the largest drop (30.3 $\rightarrow$ 19.3 TPS; 4.4 $\times \rightarrow$ 2.8 $\times$), confirming it as the key signal. Entropy and momentum provide smaller but consistent gains (w.o. entropy: 4.1 $\times$; w.o. momentum: 4.3 $\times$). Removing *position* slightly improves speed (31.1 TPS, 4.5 $\times$) but harms quality (78.5% accuracy), indicating position mainly prevents premature fixing for later tokens. Overall, confidence is essential; entropy/momentum are complementary; position improves accuracy.

Table 12. Sensitivity to CE hyper-parameters.

(a) Coverage threshold τ .			(b) Max horizon H .		
τ	TPS $\uparrow$	Acc. $\uparrow$	H	TPS $\uparrow$	Acc. $\uparrow$
Vanilla	6.9	79.3	Vanilla	6.9	79.3
0.4	79.2	78.2	10	70.1	79.0
0.5	77.0	78.7	15	72.4	78.8
0.6	77.3	78.8	20	77.3	78.8
0.7	75.2	78.8	25	78.0	78.2
0.8	74.9	78.9	30	78.2	77.9

Table 13. A comparison of our confidence extrapolator method with different decoding controllers using LLaDA-8B-Instruct model on GSM8K benchmark at generation length of 256 tokens.

Methods	TPS $\uparrow$	Speed $\uparrow$	Acc.
TSPD	30.3	4.4 $\times$	79.5
TSPD & CE	34.2	5.0 $\times$	79.4
Learn2PD	29.0	4.2 $\times$	79.1
Learn2PD & CE	32.4	4.7 $\times$	79.3
Fast-dLLM	53.8	7.8 $\times$	78.5
Fast-dLLM & CE	64.2	9.3$\times$	78.8
CD	54.5	7.9 $\times$	78.7
CD & CE	58.0	8.4 $\times$	78.7

Confidence Extrapolation with Different Controllers. Table 13 shows CE consistently improves efficiency across controllers. For TSPD, CE increases throughput from 30.3 to 34.2 TPS and speedup from 4.4 $\times$ to 5.0 $\times$ with similar quality; Learn2PD and heuristics used in Fast-dLLM exhibit similar gains. This indicates CE is controller-agnostic and complements existing stopping policies by supplying forward-looking confidence.

5. Conclusion

In this work, we address dLLM inference inefficiency and unreliability caused by repetitive denoising and step-local stopping criteria. We propose Temporal-Spatial Parallel Decoding (TSPD), a lightweight trace-aware controller that uses token-wise trajectories and token position to decide when tokens have converged and can be fixed. We further introduce Confidence Extrapolation (CE), a training-free state-space plug-in that forecasts future confidences with uncertainty to enable proactive acceleration. Across four benchmarks on LLaDA-8B-Instruct (GSM8K, MATH, HumanEval, MBPP), TSPD delivers consistent and distinguished throughput gains at different generation settings with negligible accuracy change. Our approach also composes with KV caching for additional speedups, indicating strong compatibility with system optimizations.

Impact Statement

This work enhances inference efficiency of diffusion-based large language models (dLLMs) via lightweight, training-free techniques (TSPD and Confidence Extrapolation), reducing latency and computation while preserving output quality. This facilitates practical deployment in resource-limited settings like edge devices, real-time applications, and large-scale systems.

Positive impacts include lower energy consumption for sustainable AI and greater accessibility to high-quality generation in education, scientific reasoning, coding, and productivity tools, democratizing advanced generative AI.

Faster inference may heighten misuse risks (e.g., rapid misinformation or harmful content). However, our method introduces no new vulnerabilities and is fully compatible with existing safeguards (alignment, filtering, watermarking). Responsible deployment practices should adequately mitigate concerns. Overall, the efficiency gains are predominantly beneficial for machine learning and real-world applications.

References

- Anonymous. ES-dLLM: Efficient inference for diffusion large language models by early-skipping. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=O2WvMkJbws>.
- Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and Van Den Berg, R. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993, 2021a.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Bao, W., Chen, Z., Xu, D., and Shang, Y. Learning to parallel: Accelerating diffusion large language models via learnable parallel decoding. *arXiv preprint arXiv:2509.25188*, 2025.
- Campbell, A., Benton, J., De Bortoli, V., Rainforth, T., Deligiannidis, G., and Doucet, A. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.
- Chen, K., Liu, Z., Tao, X., Liu, H., Fu, X., Zhang, S., Tu, D., Kong, L., Liu, R., and Li, H. Beyond confidence: Adaptive and coherent decoding for diffusion language models. *arXiv preprint arXiv:2512.02044*, 2025a.
- Chen, M. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Z., Fang, G., Ma, X., Yu, R., and Wang, X. dparallel: Learnable parallel decoding for dllms. *arXiv preprint arXiv:2509.26488*, 2025b.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Feng, Z., Ma, J., Chersoni, E., Zhao, X., and Bao, X. Learning to look at the other side: A semantic probing study of word embeddings in llms with enabled bidirectional attention. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025.
- Gong, S., Agarwal, S., Zhang, Y., Ye, J., Zheng, L., Li, M., An, C., Zhao, P., Bi, W., Han, J., et al. Scaling diffusion language models via adaptation from autoregressive models. *arXiv preprint arXiv:2410.17891*, 2024.
- He, Z., Sun, T., Tang, Q., Wang, K., Huang, X.-J., and Qiu, X. Diffusionbert: Improving generative masked language models with diffusion models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 4521–4534, 2023.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Hoogeboom, E., Nielsen, D., Jaini, P., Forré, P., and Welling, M. Argmax flows and multinomial diffusion: Learning categorical distributions. *Advances in neural information processing systems*, 34:12454–12465, 2021.
- Hu, Z., Meng, J., Akhauri, Y., Abdelfattah, M. S., Seo, J.-s., Zhang, Z., and Gupta, U. Accelerating diffusion language model inference via efficient kv caching and guided diffusion. *arXiv preprint arXiv:2505.21467*, 2025.

- Huang, P., Liu, S., Liu, Z., Yan, Y., Wang, S., Chen, Z., and Xiao, T. Pc-sampler: Position-aware calibration of decoding bias in masked diffusion models. *arXiv preprint arXiv:2508.13021*, 2025.
- Israel, D., Broeck, G. V. d., and Grover, A. Accelerating diffusion llms via adaptive parallel decoding. *arXiv preprint arXiv:2506.00413*, 2025.
- Kalman, R. E. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1): 35–45, 1960.
- Khanna, S., Kharbanda, S., Li, S., Varma, H., Wang, E., Birnbaum, S., Luo, Z., Miraoui, Y., Palrecha, A., Ermon, S., et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 1, 2025.
- Kong, F., Zhang, J., Liu, Y., Wu, Z., Tian, Y., Zhou, G., et al. Accelerating diffusion llm inference via local determinism propagation. *arXiv preprint arXiv:2510.07081*, 2025.
- Li, P., Zhou, Y., Muhtar, D., Yin, L., Yan, S., Shen, L., Liang, Y., Vosoughi, S., and Liu, S. Diffusion language models know the answer before decoding. *arXiv preprint arXiv:2508.19982*, 2025.
- Liu, A., Liu, J., Pan, Z., He, Y., Haffari, G., and Zhuang, B. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems*, 37:139997–140031, 2024.
- Liu, Z., Yang, Y., Zhang, Y., Chen, J., Zou, C., Wei, Q., Wang, S., and Zhang, L. dllm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025.
- Lou, A., Meng, C., and Ermon, S. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- Ma, Y., Du, L., Wei, L., Chen, K., Xu, Q., Wang, K., Feng, G., Lu, G., Liu, L., Qi, X., et al. dinfer: An efficient inference framework for diffusion language models. *arXiv preprint arXiv:2510.08666*, 2025.
- Mo, Y., Chen, Q., Li, M., Wei, Z., and Wang, Y. Decoding large language diffusion models with foreseeing movement. *arXiv preprint arXiv:2512.04135*, 2025.
- Nichol, A. Q. and Dhariwal, P. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pp. 8162–8171. PMLR, 2021.
- Nie, S., Zhu, F., Du, C., Pang, T., Liu, Q., Zeng, G., Lin, M., and Li, C. Scaling up masked diffusion models on text. *arXiv preprint arXiv:2410.18514*, 2024.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Sahoo, S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.
- Schuurmans, D., Dai, H., and Zanini, F. Autoregressive large language models are computationally universal. *arXiv preprint arXiv:2410.03170*, 2024.
- Seo, M., Kembhavi, A., Farhadi, A., and Hajishirzi, H. Bidirectional attention flow for machine comprehension. *arXiv preprint arXiv:1611.01603*, 2016.
- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- Song, Y. and Ermon, S. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.
- Wang, K., Jiang, Z., Feng, H., Zhao, W., Liu, L., Li, J., Lan, Z., and Lin, W. Creditdecoding: Accelerating parallel decoding in diffusion large language models with trace credits. *arXiv preprint arXiv:2510.06133*, 2025a.
- Wang, W., Fang, B., Jing, C., Shen, Y., Shen, Y., Wang, Q., Ouyang, H., Chen, H., and Shen, C. Time is a feature: Exploiting temporal dynamics in diffusion language models. *arXiv preprint arXiv:2508.09138*, 2025b.
- Wang, X., Xu, C., Jin, Y., Jin, J., Zhang, H., and Deng, Z. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing. *arXiv preprint arXiv:2508.09192*, 2025c.

- Wei, Q., Zhang, Y., Liu, Z., Liu, D., and Zhang, L. Accelerating diffusion large language models with slowfast sampling: The three golden principles. *arXiv preprint arXiv:2506.10848*, 2025.
- Wibisono, K. C. and Wang, Y. Bidirectional attention as a mixture of continuous word experts. In *Uncertainty in Artificial Intelligence*, pp. 2271–2281. PMLR, 2023.
- Wu, C., Zhang, H., Xue, S., Diao, S., Fu, Y., Liu, Z., Molchanov, P., Luo, P., Han, S., and Xie, E. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025a.
- Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025b.
- Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Zhao, S., Gupta, D., Zheng, Q., and Grover, A. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.
- Zhu, F., You, Z., Xing, Y., Huang, Z., Liu, L., Zhuang, Y., Lu, G., Wang, K., Wang, X., Wei, L., et al. Llada-moe: A sparse moe diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025.

A. Implementation and Training Details

A.1. Software and Hardware

We implement all methods in PyTorch (v2.1) with CUDA (v12.1) and run inference in FP16. Experiments are executed on NVIDIA A100 80GB GPUs. Unless noted otherwise, we use batch size 1 and greedy decoding to avoid confounding factors from sampling. Throughput is reported as `tokens/sec` (TPS) computed by $\text{TPS} = \frac{B \cdot L_{\text{gen}}}{T_{\text{wall}}}$, where B is the batch size and T_{wall} is the end-to-end wall-clock time from the first denoising step to termination, including remasking, controller overhead, and optional KV caching. For timing stability, we warm up 20 runs and report the mean over 200 samples (or the full evaluation set if smaller).

A.2. Base dLLM Decoding Configuration

We follow the default inference configuration of LLaDA-8B-Instruct. The response span has length $L_{\text{gen}}=256$ and is initialized as all [MASK] tokens. Denoising proceeds for at most $K=256$ steps. At step t , the model outputs per-position logits $\ell_i^{(t)} \in \mathbb{R}^V$ and the token distribution $p_i^{(t)}(v) = \text{softmax}(\ell_i^{(t)})_v$. We compute the top-1 prediction $\hat{y}_i^{(t)} = \arg \max_v p_i^{(t)}(v)$ and confidence $c_i^{(t)} = \max_v p_i^{(t)}(v)$, and use the standard low-confidence remasking rule. Let $\mathcal{A}^{(t)}$ be the set of *active* (unfixed) positions at step t . We remask the n_t lowest-confidence positions in $\mathcal{A}^{(t)}$, where n_t follows the default schedule used by LLaDA. Concretely, we set $n_t = \lceil \rho(t) \cdot |\mathcal{A}^{(t)}| \rceil$, $\rho(t) = \rho_{\max} \cdot \left(\frac{t}{K}\right)^\gamma$, with $\rho_{\max} = 0.5$ and $\gamma = 1$ as a representative setting, and we keep fixed tokens excluded from remasking. Decoding terminates when all response tokens are fixed or when $t = 1$.

A.3. Training Data Collection (Stage 1)

We collect supervision traces using an Extremely Greedy Parallel policy (Bao et al., 2025). For each prompt, we run the base dLLM to completion for K steps and store, for each position i and step t : (i) trace features $r_i^{(t)}$, (ii) the current top-1 token $\hat{y}_i^{(t)}$, and (iii) the final token $\hat{y}_i^{(1)}$. We then assign a binary label $y_i^{(t)} = \mathbb{I}[\hat{y}_i^{(t)} = \hat{y}_i^{(1)}]$, which indicates whether fixing at step t preserves the final outcome. To reduce storage, we store only the feature tensors and labels (not full vocabulary logits). For the FLAN subset used in Section 4.1 (2,640 prompts) with $L_{\text{gen}}=256$ and $K=256$, this yields on the order of $2,640 \times 256 \times 256$ token-step instances before filtering; we subsample active positions and steps to control dataset size.

A.4. Training Objective and Optimization (Stage 2)

We train TSPD with weighted binary cross-entropy to address label imbalance: $\mathcal{L} = -w_1 y \log \pi - w_0 (1 - y) \log(1 - \pi)$, with w_1/w_0 set by inverse class frequency on the training split. We use AdamW with learning rate 10^{-3} , weight decay 10^{-2} . We train for 5,000 epochs with early stopping on a held-out 10% validation split. To improve generalization across tasks, we randomly drop trace channels during training (dropout $p = 0.1$ on $r_i^{(t)}$) and apply small Gaussian noise to confidence-related features ($\sigma = 0.01$). The dLLM remains frozen; only TSPD parameters are updated.

A.5. Confidence Extrapolation (CE)

CE is a training-free per-token forecasting module that models the evolution of a scalar confidence margin. Let $\delta_i^{(t)} = \ell_{i,1}^{(t)} - \ell_{i,2}^{(t)}$ be the margin between top-1 and top-2 confidences at position i . We use a 2D constant-velocity state model $x_i^{(t)} = \begin{bmatrix} \delta_i^{(t)} \\ \dot{\delta}_i^{(t)} \end{bmatrix}$, $x_i^{(t-1)} = Ax_i^{(t)} + \epsilon$, $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, with Gaussian process noise $\epsilon \sim \mathcal{N}(0, Q)$ and observation $o_i^{(t)} = \delta_i^{(t)} + \eta$ with $\eta \sim \mathcal{N}(0, R)$. We run a Kalman filter to estimate $(\hat{x}_i^{(t)}, P_i^{(t)})$ and perform h -step prediction to obtain a conservative margin forecast $\hat{\delta}_{i,h}^{(t)} - z\sqrt{\text{Var}(\delta_{i,h}^{(t)})}$, where z controls risk sensitivity. This value is mapped to a conservative confidence proxy $\tilde{c}_i^{(t)}$ via a monotone transform (e.g., sigmoid calibration) and used as an additional trace signal and/or to choose a safe look-ahead horizon h . CE adds only scalar operations per token per step and is negligible compared to the base model forward pass.

A.6. Inference Integration

At inference step t , we run one forward pass of $\mathcal{M}$, compute trace features for active tokens, optionally run CE to obtain forecast-based signals, and apply TSPD to decide which tokens to fix. Fixed tokens are excluded from remasking and remain constant in subsequent steps. We then apply the standard remasking schedule to the remaining active tokens and proceed to step $t - 1$. We terminate when all response tokens are fixed or the step budget is exhausted.

A.7. Baselines and KV Caching

For Fast-dLLM and Credit Decoding, we enable their KV caching implementations and report their default settings. For Learn2PD and Prophet, we follow their published inference procedures without KV caching. When evaluating compatibility, we apply the same KV caching strategy to our method by caching the dLLM intermediate states used by the underlying forward passes, while keeping the controller and CE unchanged. This isolates the gains from TSPD and CE and verifies orthogonality to system-level reuse.